Dziedziczenie

Dziedziczenie

Mechanizm dziedziczenia umożliwia tworzenie hierarchii klas. Ułatwia tworzenie klas „bardziej wyspecjalizowanych”. Np. po stworzeniu klasy `Osoba` (przechowującej np. informacje o imieniu, nazwisku, numerze PESEL, umożliwiającej np. zmianę imienia, obliczenie wieku) możemy użyć jej do stworzenia bardziej wyspecjalizowanej klasy `Pracownik`. Pracownik jest osobą więc obiekty tej klasy przechowują te same informacje oraz umożliwiają wykonanie tych samych operacji co obiekty klasy `Osoba`. Możliwe jest dodanie dodatkowych danych i operacji (lub zmiana sposobu działania operacji z klasy `Osoba`). Podobnie możemy stworzyć np. klasę `Student` (student jest osobą) lub `Lekarz` (lekarz jest zarówno osobą jak i pracownikiem) czy `Nauczyciel` (też osoba i pracownik).

Efektem może być poniższa hierarchia:

```
Osoba
├── Student
└── Pracownik
    ├── Lekarz
    └── Nauczyciel
```

Mówimy, że klasa `Student` **dziedziczy** po klasie `Osoba`. Klasa `Osoba` jest **klasą bazową**/**nadklasą** (ang. superclass) klasy `Student`. Klasa `Student` jest **klasą pochodną**/**podklasą** (ang. subclass) klasy `Osoba`.

Terminy „nadklasa” i „podklasa” mogą kojarzyć się z matematycznymi terminami „nadzbiór” i „podzbiór”. Zbiór osób jest nadzbiorem zbioru studentów, a zbiór studentów jest podzbiorem zbioru osób.

Przykład

Zaczniemy od zdefiniowania klasy `Osoba` z właściwościami do odczytu `imie`, `nazwisko`, właściwością do odczytu i zapisu `wiek` oraz z metodą `pelnoletni`.

In [2]:
```python
class Osoba:
    def __init__(self, imie, nazwisko, wiek):
        if not isinstance(imie, str):
            raise TypeError("Imię musi być typu str")
        if not isinstance(nazwisko, str):
            raise TypeError("Nazwisko musi być typu str")
        self._imie = imie
        self._nazwisko = nazwisko
```

```
        self._wiek = wiek

    def __repr__(self):
        return f"Osoba({self.imie!r}, {self.nazwisko!r}, {self.wiek!r})"

    @property
    def imie(self):
        return self._imie

    @property
    def nazwisko(self):
        return self._nazwisko

    @property
    def wiek(self):
        return self._wiek

    @wiek.setter
    def wiek(self, w):
        self._wiek = w

    def pelnoletni(self):
        return self._wiek >= 18
```

In [3]:
```
osoba = Osoba("Jan", "Kowalski", 17)
```

In [4]:
```
osoba
```

Out[4]:
```
Osoba('Jan', 'Kowalski', 17)
```

In [5]:
```
osoba.imie
```

Out[5]:
```
'Jan'
```

In [6]:
```
osoba.nazwisko
```

Out[6]:
```
'Kowalski'
```

In [7]:
```
osoba.wiek
```

Out[7]:
```
17
```

In [8]:
```
osoba.pelnoletni()
```

Out[8]:
```
False
```

In [9]:
```
osoba.wiek = 18
```

In [10]:
```
osoba.wiek
```

Out[10]:
```
18
```

In [11]:
```
osoba.pelnoletni()
```

Out[11]: True

Jeżeli chcemy stworzyć klasę dziedziczącą to nazwę klasy bazowej podajemy w nawiasach po nazwie klasy pochodnej. Np. poniższa definicja:

In [12]:
```
class Pracownik(Osoba):
    pass
```

oznacza stworzenie klasy `Pracownik`, której obiekty zachowują się prawie dokładnie tak samo jak obiekty klasy `Osoba`. Wszystkie składowe (pola, metody, właściwości) zostały odziedziczone.

In [13]:
```
pracownik = Pracownik("Jan", "Kowalski", 30)
```

In [14]:
```
pracownik.imie
```

Out[14]: 'Jan'

In [15]:
```
pracownik.pelnoletni()
```

Out[15]: True

Nazwy rozpoczynające się podkreśleniem są niepubliczne z konwencji. Technicznie można je odczytać, ale kod korzystający z klasy nie powinien od nich zależeć:

In [16]:
```
pracownik._imie
```

Out[16]: 'Jan'

Metoda `__repr__` również została odziedziczona:

In [17]:
```
pracownik
```

Out[17]: Osoba('Jan', 'Kowalski', 30)

Możemy **przesłonić**/**nadpisać**/**przedefiniować** (ang. *override*) odziedziczone metody oraz dodawać nowe składowe. W poniższej definicji klasy `Pracownik` przesłonięta jest metoda `__repr__` oraz dodane są pola `_pensja`, właściwość `pensja` oraz metoda `roczne_zarobki`.

In [18]:
```
class Pracownik(Osoba):
    def __init__(self, imie, nazwisko, wiek, pensja):
        super().__init__(imie, nazwisko, wiek)
        self._pensja = pensja

    def __repr__(self):
        return (
            f"Pracownik({self.imie!r}, {self.nazwisko!r}, "
```

```
            f"{self.wiek!r}, {self.pensja!r})"
        )

    @property
    def pensja(self):
        return self._pensja

    def roczne_zarobki(self):
        return self._pensja * 12
```

Konstrukcja `super()` (użyta powyżej w konstruktorze) umożliwia odwołanie do składowych klasy bazowej. Wywołanie metody `__init__` klasy bazowej w metodzie `__init__` klasy pochodnej jest jedną z sytuacji, w których powinno się jawnie wywołać metodę specjalną. Zazwyczaj metoda `__init__` wywoływana jest podczas tworzenia obiektu, ale w tym przypadku obiekt został już utworzony.

In [19]:
```
pracownik = Pracownik("Jan", "Kowalski", 30, 3000)
```

Użycie nadpisanej metody `__repr__`:

In [20]:
```
pracownik
```

Out[20]:
```
Pracownik('Jan', 'Kowalski', 30, 3000)
```

In [21]:
```
pracownik.imie
```

Out[21]:
```
'Jan'
```

In [22]:
```
pracownik.pelnoletni()
```

Out[22]:
```
True
```

Użycie dodanych składowych:

In [23]:
```
pracownik.pensja
```

Out[23]:
```
3000
```

In [24]:
```
pracownik.roczne_zarobki()
```

Out[24]:
```
36000
```

Funkcja `isinstance`

Typ obiektu możemy sprawdzić za pomocą funkcji `type`:

In [25]:
```
type(pracownik)
```

Out[25]:
```
__main__.Pracownik
```

In [26]:
```
type(pracownik) is Pracownik
```

Out[26]:
```
True
```

Pracownik jest osobą (wszędzie tam gdzie możemy użyć obiektu klasy `Osoba` powinniśmy móc użyć obiektu klasy `Pracownik`). Niestety nie możemy tego sprawdzić za pomocą funkcji `type`:

In [27]:
```
type(pracownik) is Osoba
```

Out[27]:
```
False
```

Do sprawdzenia czy typem obiektu jest zadana klasa lub jedna z jej klas pochodnych służy funkcja `isinstance`:

In [28]:
```
isinstance(pracownik, Pracownik)
```

Out[28]:
```
True
```

In [29]:
```
isinstance(pracownik, Osoba)
```

Out[29]:
```
True
```

Jeżeli chcemy np. sprawdzić czy argument funkcji jest odpowiedniego typu, to zazwyczaj lepszym pomysłem będzie użycie `isinstance` zamiast sprawdzenia typu funkcją `type`.

Rozważmy poniższą funkcję:

In [30]:
```
def info(osoba):
    if type(osoba) is not Osoba:
        raise TypeError("Argument złego typu")
    print(f"imię: {osoba.imie}")
    print(f"nazwisko: {osoba.nazwisko}")
    print(f"wiek: {osoba.wiek}")
```

In [31]:
```
info(osoba)
```

```
imię: Jan
nazwisko: Kowalski
wiek: 18
```

Funkcja `info` odrzuci jednak obiekt `pracownik`, mimo że jest on osobą. Sprawdza dokładny typ, więc dla podklasy `Pracownik` zgłosi `TypeError`:

In [32]:
```
info(pracownik)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[32], line 1
----> 1 info(pracownik)

Cell In[30], line 3, in info(osoba)
      1 def info(osoba):
      2     if type(osoba) is not Osoba:
----> 3         raise TypeError("Argument złego typu")
      4     print(f"imię: {osoba.imie}")
      5     print(f"nazwisko: {osoba.nazwisko}")
      6     print(f"wiek: {osoba.wiek}")

TypeError: Argument złego typu
```

Sprawdzenie typu powinno raczej wyglądać tak:

In [33]:
```
def info(osoba):
    if not isinstance(osoba, Osoba):
        raise TypeError("Argument złego typu")
    print(f"imię: {osoba.imie}")
    print(f"nazwisko: {osoba.nazwisko}")
    print(f"wiek: {osoba.wiek}")
```

In [34]:
```
info(osoba)
```

```
imię: Jan
nazwisko: Kowalski
wiek: 18
```

In [35]:
```
info(pracownik)
```

```
imię: Jan
nazwisko: Kowalski
wiek: 30
```

Funkcja `isinstance` może być użyta również do obiektów typów wbudowanych:

In [36]:
```
isinstance([1, 2, 3], list)
```

Out[36]: True

In [37]:
```
isinstance(5, int)
```

Out[37]: True

In [38]:
```
isinstance({}, dict)
```

Out[38]: True

Jako drugiego argumentu można również użyć krotki typów:

In [39]:
```
liczba = 3
```

In [40]:
```
isinstance(liczba, (int, float))
```

Out[40]:
```
True
```

Dziedziczenie a hermetyzacja

W klasie pochodnej nie powinno się korzystać ze składowych niepublicznych klasy bazowej.

Niech klasa `PracownikZPremia` umożliwia przyznawanie pracownikom premii. Pracownikom można przyznawać premie, a ich roczne zarobki, to suma miesięcznych pensji i przyznanych premii. Zwróć uwagę na definicję metody `roczne_zarobki` (zostało tam użyte niepubliczne pole `_pensja` z klasy `Pracownik`):

Dla wygody powtarzamy obie klasy potrzebne do przykładu premii. Ich publiczny interfejs pozostaje taki sam:

In [41]:
```
class Osoba:
    def __init__(self, imie, nazwisko, wiek):
        if not isinstance(imie, str):
            raise TypeError("Imię musi być typu str")
        if not isinstance(nazwisko, str):
            raise TypeError("Nazwisko musi być typu str")
        self._imie = imie
        self._nazwisko = nazwisko
        self._wiek = wiek

    def __repr__(self):
        return f"Osoba({self.imie!r}, {self.nazwisko!r}, {self.wiek!r})"

    @property
    def imie(self):
        return self._imie

    @property
    def nazwisko(self):
        return self._nazwisko

    @property
    def wiek(self):
        return self._wiek

    @wiek.setter
    def wiek(self, w):
        self._wiek = w

    def pelnoletni(self):
        return self._wiek >= 18
```

In [42]:
```
class Pracownik(Osoba):
    def __init__(self, imie, nazwisko, wiek, pensja):
        super().__init__(imie, nazwisko, wiek)
        self._pensja = pensja

    def __repr__(self):
        return (
            f"Pracownik({self.imie!r}, {self.nazwisko!r}, "
            f"{self.wiek!r}, {self.pensja!r})"
        )

    @property
    def pensja(self):
        return self._pensja

    def roczne_zarobki(self):
        return self._pensja * 12
```

In [43]:
```
class PracownikZPremia(Pracownik):
    def __init__(self, imie, nazwisko, wiek, pensja):
        super().__init__(imie, nazwisko, wiek, pensja)
        self._premia = 0

    def przyznaj_premie(self, premia):
        self._premia += premia

    def roczne_zarobki(self):
        return self._pensja * 12 + self._premia
```

In [44]:
```
pracownik = PracownikZPremia("Jan", "Kowalski", 30, 3000)
```

In [45]:
```
pracownik.przyznaj_premie(4000)
```

In [46]:
```
pracownik.roczne_zarobki()
```

Out[46]:
```
40000
```

Załóżmy, że chcemy teraz zmodyfikować implementację klasy `Pracownik`. Zamiast przechowywać pensję chcemy przechowywać sumę dwunastu pensji, a pensję obliczać w razie potrzeby. Modyfikacja taka nie powinna powodować żadnych zmian w kodzie używającym tej klasy:

In [47]:
```
class Pracownik(Osoba):
    def __init__(self, imie, nazwisko, wiek, pensja):
        super().__init__(imie, nazwisko, wiek)
        self._roczne_zarobki = pensja * 12

    def __repr__(self):
        return (
            f"Pracownik({self.imie!r}, {self.nazwisko!r}, "
            f"{self.wiek!r}, {self.pensja!r})"
        )
```

```
    @property
    def pensja(self):
        return self._roczne_zarobki / 12

    def roczne_zarobki(self):
        return self._roczne_zarobki
```

Niestety teraz przestanie działać metoda `roczne_zarobki` z klasy `PracownikZPremia`:

In [48]:
```
class PracownikZPremia(Pracownik):
    def __init__(self, imie, nazwisko, wiek, pensja):
        super().__init__(imie, nazwisko, wiek, pensja)
        self._premia = 0

    def przyznaj_premie(self, premia):
        self._premia += premia

    def roczne_zarobki(self):
        return self._pensja * 12 + self._premia
```

In [49]:
```
pracownik = PracownikZPremia("Jan", "Kowalski", 30, 3000)
```

Wywołanie `roczne_zarobki` próbuje nadal odczytać pole `_pensja`, którego nowa wersja klasy bazowej już nie tworzy. Otrzymamy więc `AttributeError`:

In [50]:
```
pracownik.roczne_zarobki()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[50], line 1
----> 1 pracownik.roczne_zarobki()

Cell In[48], line 10, in PracownikZPremia.roczne_zarobki(self)
      9     def roczne_zarobki(self):
---> 10         return self._pensja * 12 + self._premia

AttributeError: 'PracownikZPremia' object has no attribute '_pensja'
```

Jest to spowodowane użyciem składowej niepublicznej. Żeby uniknąć tego problemu możemy w metodzie `roczne_zarobki` klasy `PracownikZPremia` użyć metody `roczne_zarobki` z jej klasy bazowej `Pracownik`:

In [51]:
```
class PracownikZPremia(Pracownik):
    def __init__(self, imie, nazwisko, wiek, pensja):
        super().__init__(imie, nazwisko, wiek, pensja)
        self._premia = 0

    def przyznaj_premie(self, premia):
        self._premia += premia

    def roczne_zarobki(self):
        return super().roczne_zarobki() + self._premia
```

Podobnie jak w konstruktorze, tutaj również użyliśmy konstrukcji `super()`.

In [52]:
```
pracownik = PracownikZPremia("Jan", "Kowalski", 30, 3000)
```

In [53]:
```
pracownik.roczne_zarobki()
```

Out[53]:
```
36000
```

Klasa `object`

Wszystkie klasy dziedziczą bezpośrednio lub pośrednio po klasie `object`:

In [54]:
```
isinstance(pracownik, object)
```

Out[54]:
```
True
```

In [55]:
```
isinstance(5, object)
```

Out[55]:
```
True
```

Jeżeli nie podamy klasy bazowej w definicji klasy, to domyślnie jest to klasa `object`, czyli np. deklaracja

In [56]:
```
class A:
    pass
```

jest równoważna tej:

In [57]:
```
class A(object):
    pass
```

Domyślne implementacje np. metod `__str__`, `__repr__` czy `__eq__` lub `__lt__` są w klasach odziedziczone właśnie po klasie `object`.